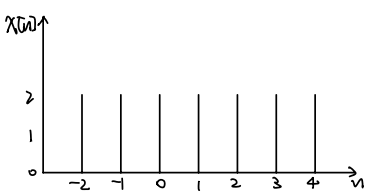
CSC420 A1

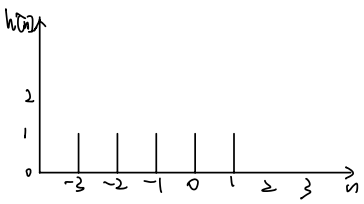
Xin-tong Zhang 1006868660

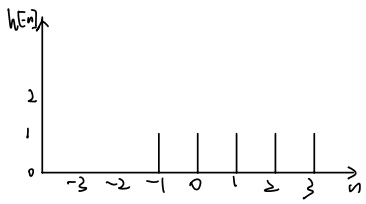
Q1. a $x[n] = \begin{cases} 2 & -2 \le n \le 4 \\ 0 & o/w \end{cases}$ $h[n] = \begin{cases} 1 & -3 \le n \le 1 \\ 0 & o/w \end{cases}$

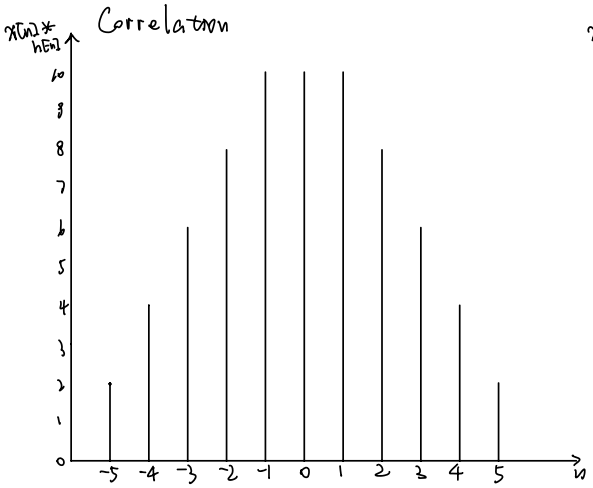
correlation: $y[n] = x[n] * h[-n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k]$

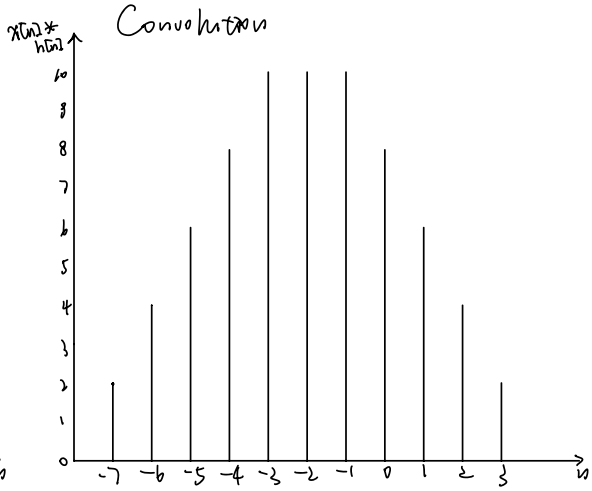
convolution: $z[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[k+n]$









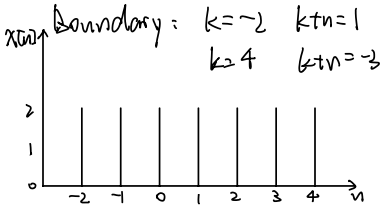


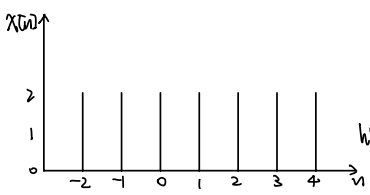
Boundary: $k = -2$ $-(n-k) = 3$ $n-k = -3$ $n = -5$

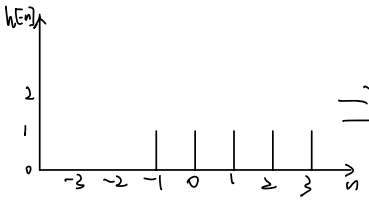
$k = 4$ $-(n-k) = -1$ $n-k = 1$ $n = 5$

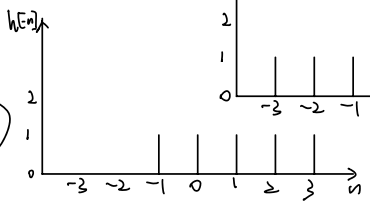
Boundary: $k = -2$ $k+n = 1$ $n = 3$

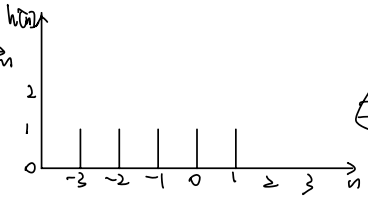
$k = 4$ $k+n = -3$ $n = -7$

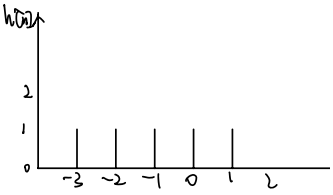






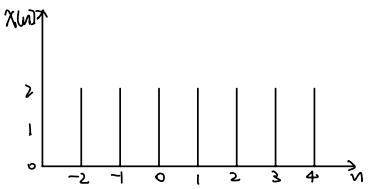


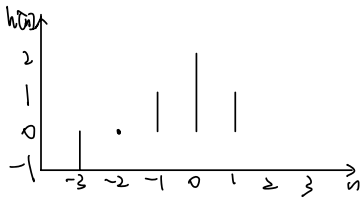


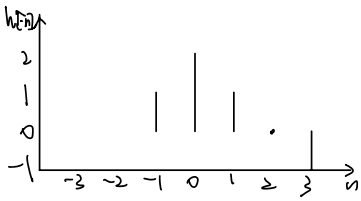


b.

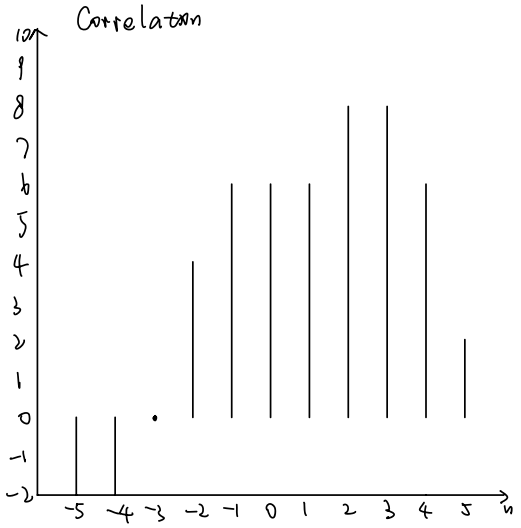
$$x[n] = \begin{cases} 2 & -2 \le n \le 4 \\ 0 & o/w \end{cases} \qquad h[n] = \begin{cases} 2 - |n| & -3 \le n \le 1 \\ 0 & o/w \end{cases}$$





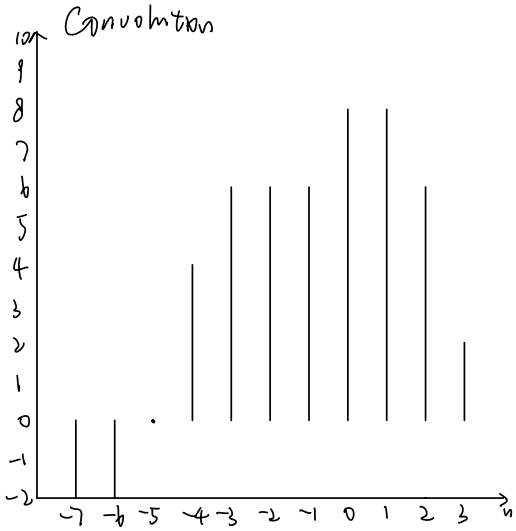


Correlation



same boundary as a from $n=-5$ to $n=5$

Convolution



$n=-7$ to $n=3$

Q2. Let $u \in R^{m+1}$, $v \in R^{n+1}$, assume $m > n$

$p_1 = u_m x^m + u_{m-1} x^{m-1} + \cdots + u_1 x + u_0$

$p_2 = v_n x^n + v_{n-1} x^{n-1} + \cdots + v_1 x + v_0$

Then, add zero-paddings to make them into R^{m+n+1}

$u = (0, 0, 0 \cdots 0, u_m, u_{m-1}, \cdots u_1, u_0) \in R^{m+n+1}$

$v = (0, 0, 0 \cdots 0, v_n, v_{n-1}, \cdots v_1, v_0) \in R^{m+n+1}$

Then, let $p_1 p_2 = \sum_{i=0}^{m+n} c_i x^i$

$$c_i x^i = u_0 x^0 \cdot v_i x^i + u_1 x^1 \cdot v_{i-1} x^{i-1} + \cdots + u_i x^i \cdot v_0 x^0$$
$$= \sum_{j=0}^{i} u_j v_{i-j} x^i$$
$$= (u \cdot v)_i x^i$$

Thus, $c = u \cdot v$

Coefficients of multiplication is convolution of u and v

Q3. $\Delta I = I_{xx} + I_{yy}$

Suppose we rotate (x,y) by θ to get (r,r')

$$r = x\cos\theta - y\sin\theta \;,\quad r' = x\sin\theta + y\cos\theta$$
$$x = r\cos\theta \quad y = r\sin\theta$$
$$\frac{\partial r}{\partial x} = \cos\theta \quad \frac{\partial r}{\partial y} = \sin\theta$$
$$\frac{\partial r'}{\partial x} = -r\sin\theta \quad \frac{\partial r'}{\partial y} = r\cos\theta$$

$$\frac{\partial I}{\partial x} = \frac{\partial r}{\partial x}\frac{\partial I}{\partial r} + \frac{\partial r'}{\partial x}\frac{\partial I}{\partial r'} = \frac{\partial I}{\partial r}\cos\theta + \frac{\partial I}{\partial r'}\sin\theta$$

$$\frac{\partial I}{\partial y} = \frac{\partial r}{\partial y}\frac{\partial I}{\partial r} + \frac{\partial r'}{\partial y}\frac{\partial I}{\partial r'} = -\frac{\partial I}{\partial r}\sin\theta + \frac{\partial I}{\partial r'}\cos\theta$$

Then, we can get second partial derivatives

$$\frac{\partial^2 I}{\partial x^2} = \frac{\partial}{\partial x}\left[\frac{\partial r}{\partial x}\frac{\partial I}{\partial r} + \frac{\partial r'}{\partial x}\frac{\partial I}{\partial r'}\right]$$
$$= \frac{\partial r}{\partial x}\frac{\partial}{\partial r}\frac{\partial I}{\partial x}\cos\theta + \frac{\partial r'}{\partial x}\frac{\partial}{\partial r'}\frac{\partial I}{\partial x}\sin\theta$$
$$= \frac{\partial r}{\partial x}\left(\frac{\partial}{\partial r}\frac{\partial I}{\partial r}\cos\theta + \frac{\partial r'}{\partial r}\frac{\partial}{\partial r'}\frac{\partial I}{\partial r}\sin\theta\right)\cos\theta + \frac{\partial r'}{\partial x}\left(\frac{\partial}{\partial r}\frac{\partial I}{\partial r}\cos\theta + \frac{\partial}{\partial r'}\frac{\partial I}{\partial r}\sin\theta\right)\sin\theta$$
$$= \frac{\partial^2 I}{\partial r^2}\cos^2\theta + \frac{\partial^2 I}{\partial r\partial r'}2\sin\theta\cos\theta + \frac{\partial^2 I}{\partial r'^2}\sin^2\theta$$
$$\frac{\partial^2 I}{\partial y^2} = \frac{\partial^2 I}{\partial r^2}\sin^2\theta - \frac{\partial^2 I}{\partial r\partial r'}2\sin\theta\cos\theta + \frac{\partial^2 I}{\partial r'^2}\cos^2\theta$$

Thus, $\frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2} = \frac{\partial^2 I}{\partial r^2}\cos^2\theta + \frac{\partial^2 I}{\partial r\partial r'}2\sin\theta\cos\theta + \frac{\partial^2 I}{\partial r'^2}\sin^2\theta + \frac{\partial^2 I}{\partial r^2}\sin^2\theta - \frac{\partial^2 I}{\partial r\partial r'}2\sin\theta\cos\theta$

$$= \frac{\partial^2 I}{\partial r^2}(\cos^2\theta + \sin^2\theta) + \frac{\partial^2 I}{\partial r'^2}(\sin^2\theta + \cos^2\theta)$$
$$= \frac{\partial^2 I}{\partial r^2} + \frac{\partial^2 I}{\partial r'^2}$$

i) $\Delta I = I_{xx} + I_{yy} = I_{rr} + I_{r'r'}$

ii) Laplacian is rotation invariant

Q4. J = conv2(F, I) $O(k^2n^2)$

1. conv2(G_1, conv2(F, I))

conv2(F, I) $O(k^2n^2)$

Then we convolve resulting image F*I (n×n) with second filter G_1 (k×k)

takes $O(k^2n^2)$

∴ $O(k^2n^2) + O(k^2n^2) = O(k^2n^2)$

2. conv2(conv2(G_1, F), I)

conv2(G_1, F) $G_1(k \times k)$ $F(k \times k) \Rightarrow O(k^2 \times k^2) = O(k^4)$

Then, we convolve resulting filter G*F of size (2k-1)×(2k-1) with Image I (n×n)

$O((2k-1)^2 n^2) = O(k^2 n^2)$

∴ $O(k^4) + O(k^2n^2)$

$O(k^2n^2) < O(k^4) + O(k^2n^2)$ First Approach conv2(G_1, conv2(F, I)) is more efficient since k is typically much smaller than n in image processing, $O(k^2n^2) >> O(k^4)$

However, the addition $O(k^4)$ still introduce significant overhead.

Q5. Minimum information required from the Gaussian pyramid is I_0

Suppose in each step i, we blur image I_k with operation B_i and downsample the blurred image by factor 2 with operation D_i. Define the inverse of D_i as unsample U_{i+1}

By definition of Gaussian pyramid

$$I_{k+1} = D_k B_k I_k$$

By definition of Laplacian pyramid

$$I_k - B_k I_k = L_k$$

Starting from I_n, we can reconstruct I_{n-1} then I_{n-2} and so on.

Then, we get recursion formula:

$$I_k - U_{k+1} I_{k+1} = L_k$$

$$I_k = U_{k+1} I_{k+1} + L_k$$

So we can get I_0 by this.

To express I_0 directly in terms of I_n and Laplacian levels $L_0, L_1, \cdots L_{n-1}$

$$\begin{aligned} I_0 &= U_1 I_1 + L_0 \\ &= U_1 (U_2 I_2 + L_1) + L_0 \\ &= U_1 U_2 (U_3 I_3 + L_2) + U_1 L_1 + L_0 \\ &\cdots \\ &= \prod_{i=0}^{n} U_i \; I_n + \sum_{j=0}^{n-1} \left(\prod_{k=0}^{j} U_k \right) L_j \end{aligned}$$

i) Minimum information required from Gaussian pyramid is I_0

closed expression for I_0 is

$$I_0 = \prod_{i=0}^{n} U_i \; I_n + \sum_{j=0}^{n-1} \left(\prod_{k=0}^{j} U_k \right) L_j$$

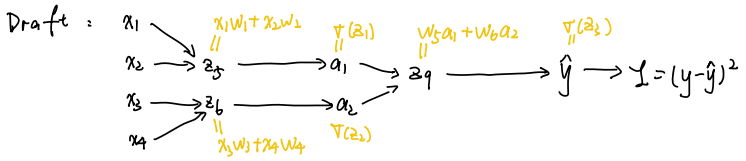
where U_i is unsample operation at step i

I_n is smallest image at level n

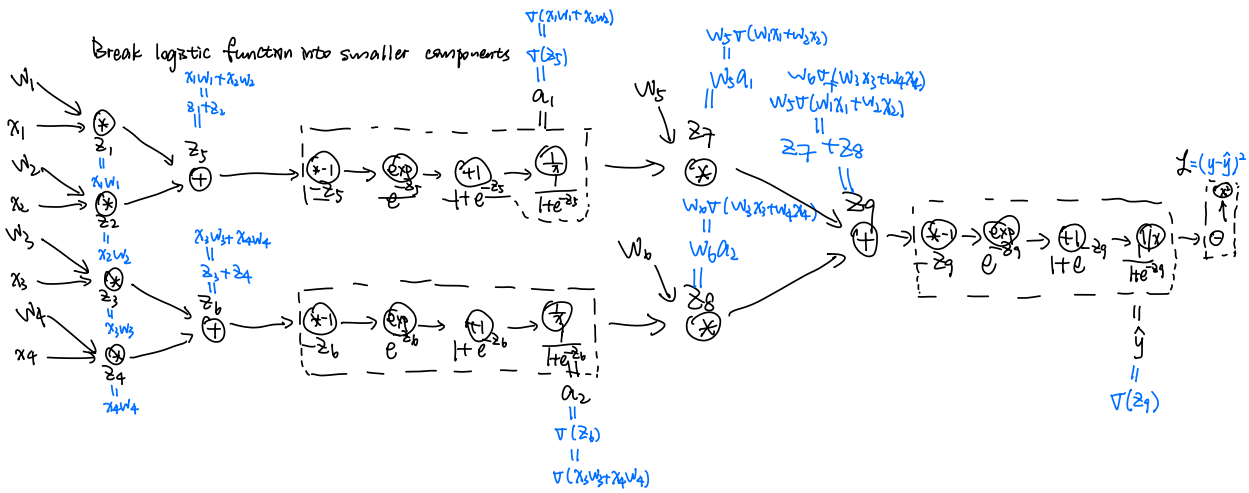
L_j is Laplacian image at level j

Q 6. a. $\hat{y} = \sigma(w_5\sigma(w_1x_1+w_2x_2)+w_6\sigma(w_3x_3+w_4x_4))$

Draft :



Break logistic function into smaller components



b. Given $(x_1, x_2, x_3, x_4) = (-1.1, 1.3, -1.5, 2.0)$ with true label 0.0

$(w_1, w_2, w_3, w_4, w_5, w_6) = (-0.2, -0.3, 0.4, 0.5, -0.4, 0.3)$

$\hat{y} = \sigma(w_5\sigma(w_1x_1+w_2x_2)+w_6\sigma(w_3x_3+w_4x_4))$

$L = (y-\hat{y})^2$

① Forward Pass

$z_5 = w_1x_1 + w_2x_2 = -0.2\times(-1.1) + (-0.3)\times 1.3 = 0.22-0.39 = -0.17$

$a_1 = \sigma(z_5) = \sigma(-0.17) = \frac{1}{1+e^{0.17}} = 0.4576$

$z_6 = w_3x_3 + w_4x_4 = 0.40\times(-1.5) + 0.50\times 2.0 = -0.6+1 = 0.4$

$a_2 = \sigma(z_6) = \sigma(0.40) = \frac{1}{1+e^{-0.40}} = 0.5987$

$z_9 = w_5a_1 + w_6a_2 = (-0.40)\times 0.4576 + 0.3\times 0.5987 = -0.0034$

$\hat{y} = \sigma(z_9) = \frac{1}{1+e^{0.0034}} = 0.4992$

② Loss $L = (y-\hat{y})^2 = (0-0.4992)^2 = 0.2492$

③ Backpropagation

$\frac{\partial L}{\partial \hat{y}} = 2(y-\hat{y}) = 2\times(0.0-0.4992) = -0.9984$

$\hat{y} = \frac{1}{1+e^{-z_9}}$

$\frac{\partial \hat{y}}{\partial z_9} = \hat{y}(1-\hat{y}) = 0.4992\times(1-0.4992) = 0.2499$

$\frac{\partial z_9}{\partial w_4} = w_6\frac{\partial a_2}{\partial w_4}$

$\frac{\partial a_2}{\partial z_6} = a_2(1-a_2) = 0.5987\times(1-0.5987) = 0.2403$

$\frac{\partial z_6}{\partial w_4} = x_4 = 2$

$\frac{\partial a_2}{\partial w_4} = \frac{\partial a_2}{\partial z_6}\cdot\frac{\partial z_6}{\partial w_4} = 0.2403\times 2.0 = 0.4806$

$\therefore \frac{\partial z_9}{\partial w_4} = w_6\times 0.4806 = 0.3\times 0.4806 = 0.1442$

$\therefore \frac{\partial L}{\partial w_4} = \frac{\partial L}{\partial \hat{y}}\cdot\frac{\partial \hat{y}}{\partial z_9}\cdot\frac{\partial z_9}{\partial w_4}$

$= -0.9984\times 0.2499\times 0.1442 = \boxed{-0.0360}$

Q7. Given convolution layer C: Input channels = 50
Size of input channels = 12x12
filters = 20
size of each filter = 4x4
padding = 1
stride = 2

C: output size = $\left\lfloor \frac{\text{Input size} - \text{Kernel size} + 2\times\text{Padding}}{\text{Stride}} \right\rfloor + 1$

$= \frac{12-4+2\times1}{2} + 1 = 6$

output of convolution layer has dimensions 6x6 and 20 feature maps

P: Output size = $\left\lfloor \frac{\text{Input size} - \text{Kernal size}}{\text{stride}} \right\rfloor + 1$

$= \frac{6-3}{1} + 1 = 4$

output of max pooling layer has dimensions 4x4 and 20 feature maps

FLOPs calculation: 3x3 max pooling layer compare 9 elements
comparisons = 3x3-1 = 8
positions of each window in each feature map = 4x4 = 16
FLOPS per feature map = 16x8 = 128
FLOPs for P = 20x128 = 2560

FLOP per filter = # multiplications + # additions
Each filter of size 4x4 performs multiplications and additions over 50 input channels
with bias: FLOP per position = 4x4x50 + 4x4x50 = 1600
FLOP per filter = 36x1600 = 57600
Total FLOP for C = 57600 x 20 = 1152000
Total FLOP for C and P = 1152000 + 2560 = **1154560**
without bias: FLOP per position = 4x4x50 + 4x4x50 - 1 = 1599
FLOP per filter = 36x1599 = 57564
Total FLOP for C = 57564 x 20 = 1151280
Total FLOP for C and P = 1151280 + 2560 = **1153840**

Q8. # trainable parameters = kernel size × # input channels × # output channels + # biases

$C_1 = 5 \times 5 \times 1 \times 6 + 6 = 156$

$C_3 = 5 \times 5 \times 6 \times 16 + 16 = 2416$

$C_5 = 5 \times 5 \times 16 \times 120 + 120 = 48120$

for fully connected layers: # trainable parameters = # input layers × # output layers + biases

F_6: $120 \times 84 + 84 = 10164$

output: $84 \times 10 + 10 = 850$

No trainable parameters for pooling layers because they only perform fixed operation on input without weights or biases

Total trainable parameters = $C_1 + C_3 + C_5 + F_6$ + output

$= 156 + 2416 + 48120 + 10164 + 850$

$= \boxed{61706}$

Q9. logistic activation function

$$f(x) = \frac{1}{1+e^{-x}} = \sigma(x)$$

Assume $z = wx + b$

output $y = \sigma(wx+b)$

$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

$$= \left(\frac{e^{-z}}{1+e^{-z}}\right)^2 \cdot \frac{dz}{dx}$$

$$= \frac{e^{-z}}{1+e^{-z}} \cdot \frac{1}{1+e^{-z}} \cdot w$$

$$= \left(1 - \frac{1}{1+e^{-z}}\right) \cdot \frac{1}{1+e^{-z}} \cdot w$$

$$= (1 - \sigma(z)) \cdot \sigma(z) \cdot w$$

$$= (1-y) \cdot y \cdot w$$

Thus, in order to compute gradient, there is no need for input for gradient as long as we have the output.

Part 2

Task 1

Overall, DBI represents a controlled, studio-quality dataset, while SDD portrays more dynamic, real-world scenarios with varied backgrounds and contexts.

The DBI dataset consists of professional, high-quality photos where dogs are the central focus, often set against plain white or neutral backgrounds with no distractions. The images have consistent lighting and clear detail, making them ideal for studies requiring precision, like breed identification. In contrast, the SDD dataset features more candid, real-world images where dogs are shown in diverse environments, such as on the beach, in cars, or with people. These photos often include other elements or subjects, capturing natural interactions and daily activities, making them appear more like images sourced from social media or personal collections

Task 2

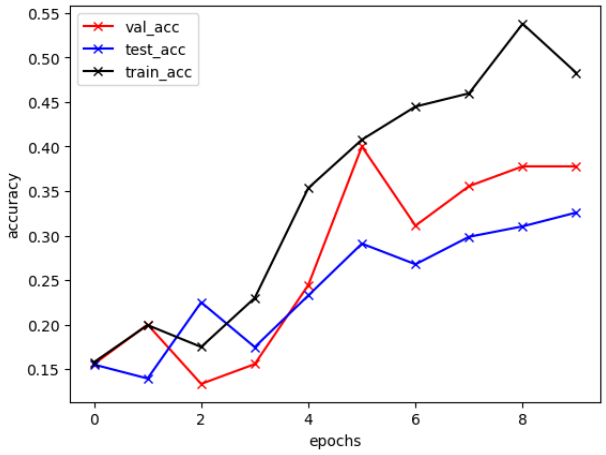


Figure 1: CNN without dropout

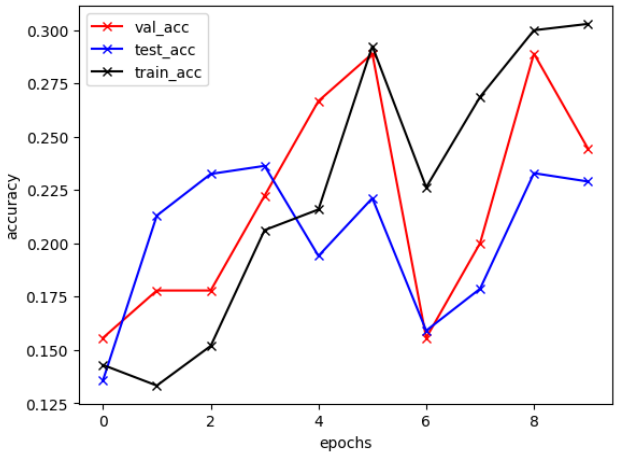


Figure 2: CNN with dropout

Figure 1 shows the accuracy of the CNN without dropout, while **Figure 2** shows the accuracy of the CNN with dropout. The test accuracy for the model without dropout is approximately 0.3368, whereas with dropout it is 0.2383. Overall, the model without dropout performs better during testing.

Both models exhibit a gradual improvement in prediction accuracy, with slight dips occurring around epochs 6 and 9. The CNN without dropout achieves significantly higher accuracy on the training set. However, the gap between training and test accuracy is much larger for the model without dropout compared to the one with dropout, indicating that dropout helps reduce overfitting. Dropout helps reduce overfitting by randomly deactivating a portion of neurons

during training, preventing the model from becoming too dependent on specific neurons. This forces the network to learn more generalized and robust features, improving its ability to generalize to unseen data and reducing the gap between training and test performance.

Task 3

3a

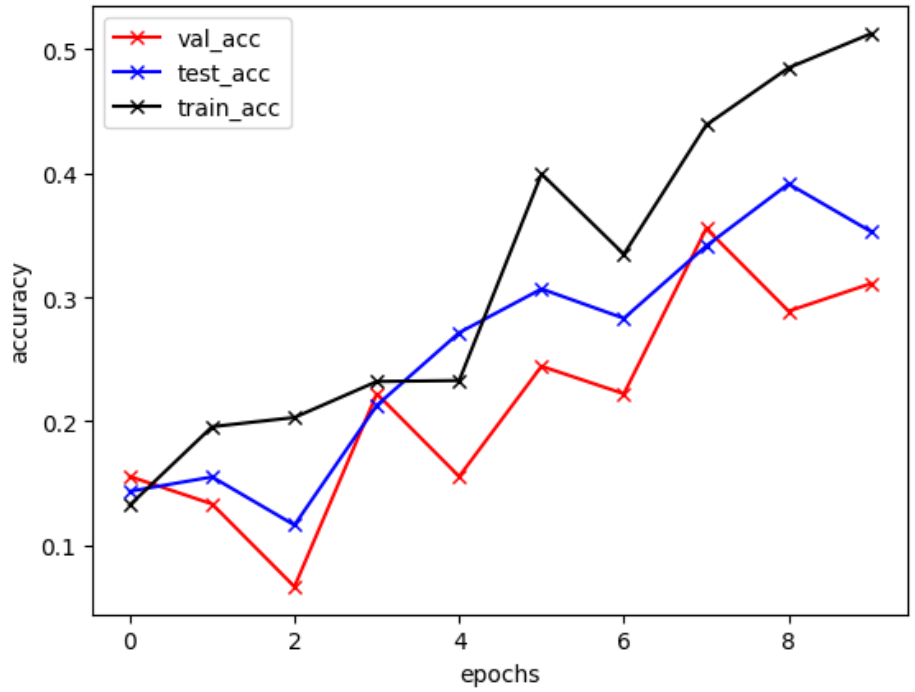


Figure 3: ResNe18 untrained on DBI

Test accuracy for ResNet18 on DBI dataset is around 0.3679. Compared with CNN model, ResNet18 has a slight improvement in accuracy and has better generalization with shorter gap between training and test sets.

3b Accuracy of above model on SDD dataset: 0.2863

It is easily observed that the model has better accuracy on the test set of DBI than the whole set of SDD.

The ResNet-18 model performs better on the DBI dataset due to its uniformity and controlled conditions, allowing the model to learn specific patterns easily. However, its generalization ability weakens on the more varied and complex SDD dataset, leading to lower accuracy.

Task 4

ResNet18 (Untrained): DBI - 0.3679, SDD - 0.2863

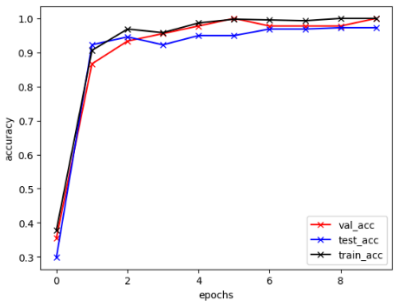
ResNet18 (Pretrained): DBI - 0.9689, SDD - 0.9035

ResNet34 (Pretrained): DBI - 0.9845, SDD - 0.9303

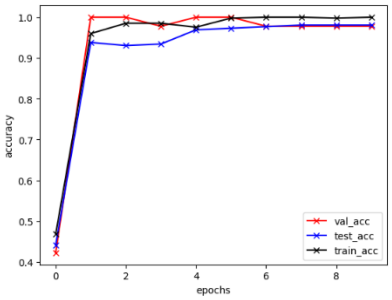
ResNeXt50 (Pretrained): DBI - 0.9741, SDD - 0.9311

SwinTransformer (Tiny): DBI - 0.9430, SDD - 0.9059

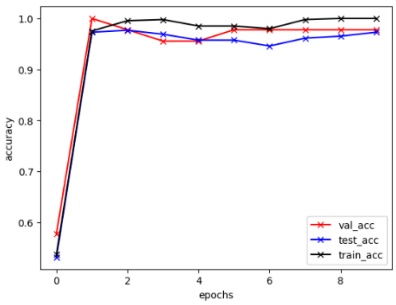
DenseNet121 (Pretrained): DBI - 0.9689, SDD - 0.9327



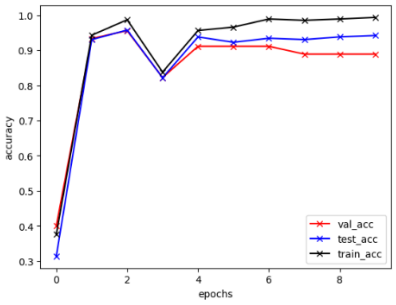
ResNet 18



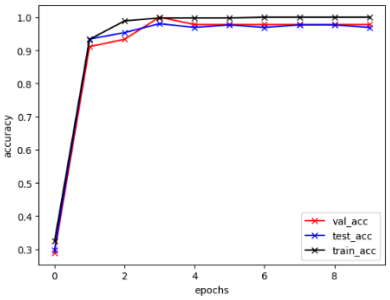
ResNet 34



ResNeXt 50



SwinTransformer (Tiny)



DenseNet121

1. **ResNet18** (Pretrained): DBI: 0.9689, SDD: 0.9035

After pre-training, ResNet18 performs much better on both datasets. However, there is a noticeable drop in accuracy when transitioning from DBI (a more formal, specific dataset) to SDD (a more diverse and complex dataset). This gap suggests that while ResNet18 is able to perform well on DBI, its ability to generalize to the more diverse SDD dataset is somewhat limited.

2. **ResNet34** (Pretrained): DBI: 0.9845, SDD: 0.9303

ResNet34, a deeper version of ResNet18, shows improved performance on both DBI and SDD. The generalization gap between DBI and SDD is smaller (about 5%), indicating that ResNet34 is

better at generalizing to more diverse datasets. This could be due to its increased depth and ability to learn more complex features.

3. **ResNeXt50** (Pretrained): DBI: 0.9741, SDD: 0.9311

ResNeXt50 performs slightly better than ResNet18 on both datasets, with a similar generalization gap to ResNet34. The strong performance on SDD suggests that ResNeXt50's architecture, which uses grouped convolutions, enables better feature extraction and improved generalization to more varied images compared to ResNet18.

4. **SwinTransformer (Tiny):** DBI: 0.9430, SDD: 0.9059

The SwinTransformer (Tiny) performs slightly worse than the convolutional networks on DBI but generalizes relatively well to SDD, with a smaller performance gap (~4%) compared to ResNet18. This shows that the transformer-based architecture may not capture the same level of specificity on DBI but is better at generalizing to more complex, varied data in SDD. This behavior aligns with transformer models' ability to capture global contextual information better than CNNs.

5. **DenseNet121** (Pretrained): DBI: 0.9689, SDD: 0.9327

DenseNet121 performs similarly to ResNet18 on DBI, but it generalizes better to SDD, with the smallest performance gap (~3.6%). DenseNet's architecture, which enables more efficient feature reuse, likely contributes to its strong generalization, as it can better adapt the learned features from DBI to SDD's more complex variations.

General Discussion:

ResNet18 (Pretrained) shows the largest gap in performance between DBI and SDD (~6.5%). This suggests that it struggles more with generalization to the more varied and complex SDD dataset compared to its deeper counterparts like ResNet34 and ResNeXt50.

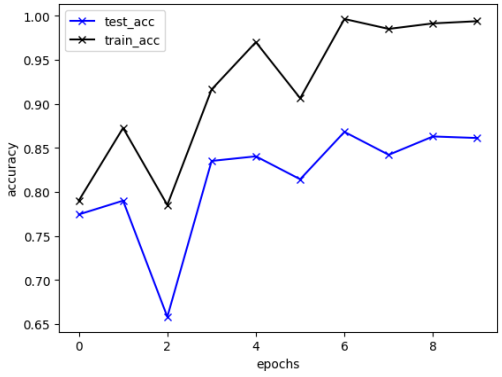
DenseNet121 and ResNeXt50 have the smallest generalization gaps (~3.6% and ~4.3%, respectively). This indicates that both models are better suited for cross-dataset generalization, possibly due to their deeper architectures (ResNeXt50) or more efficient feature reuse (DenseNet121).

While SwinTransformer does not outperform the best CNNs on DBI, it maintains a relatively small generalization gap (~4%). This suggests that the self-attention mechanism used in transformers helps in dealing with the diversity and complexity of the SDD dataset, making it competitive in terms of generalization.

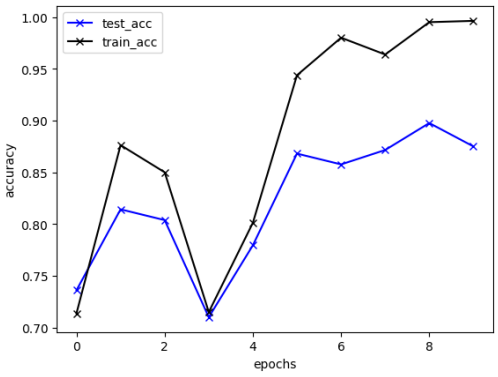
Task 5

To distinguish between images from the SDD and DBI datasets, I began by balancing the number of images in both datasets by randomly deleting some pictures from the larger SDD dataset. I then divided the data into training and test sets, using 70% for training and 30% for testing, with no validation set, as I did not require it for hyperparameter tuning.

For model selection, I chose ResNeXt50 and DenseNet121 due to their strong performance in previous exercises. DenseNet121 was ultimately selected because it had the smallest gap between training and testing performance and an overall higher accuracy. During training, I adjusted the learning rate from 0.01 to 0.012 and reduced the batch size from 64 to 32 which is to avoid memory limitations in Google Colab and it might be better if we are able to have a larger batch size.



ResNeXt 50



DenseNet121

After training for 10 epochs, DenseNet121 achieved a final accuracy of 0.8895 on the test dataset, slightly outperforming ResNeXt50, which had an accuracy of 0.8740. The final choice of DenseNet121 was based on its consistent performance and smaller generalization gap between the training and test datasets.

Task 6

1: Access to the entire DBI dataset but no access to the SDD dataset

In this case, since we only have DBI for training, we could use techniques like data augmentation to simulate some of the variability found in the SDD dataset, such as diverse lighting conditions, backgrounds, and other real-world complexities. Additionally, we could use domain generalization methods, which aim to train models that generalize well to unseen data

distributions by using techniques like domain adversarial training or feature space regularization to encourage learning more generalized features during training on DBI.

2: Access to the entire DBI dataset and a small portion (10%) of the SDD dataset

With access to a small portion of the SDD dataset, we could leverage fine-tuning. First, we would train the model on DBI, then fine-tune it using the 10% of SDD data, allowing the model to adjust to the specific characteristics of SDD. Another approach could involve semi-supervised learning techniques to augment the small SDD data, such as using pseudo-labeling or consistency regularization to further improve the model's generalization to the full SDD dataset.

3: Access to the entire DBI dataset and a small portion (10%) of the SDD dataset without labels

In this case, unsupervised domain adaptation methods could be applied. We could use techniques like adversarial domain adaptation, where a discriminator is trained to distinguish between DBI and SDD data, while the model learns features that are invariant across both domains. Additionally, self-supervised learning could be useful, where the model learns useful features from the unlabelled SDD data through tasks like contrastive learning, which helps improve the model's ability to generalize to SDD even without access to labels.

Task 7

The issues explored in this exercise, such as the difference in performance between DBI and SDD datasets, have significant implications for real-world applications, especially in terms of bias and performance generalization. When a model is trained on a dataset from one setting (e.g., a university with controlled, clean images like DBI), it may perform well in similar conditions but struggle when deployed in a completely different setting (e.g., a retirement home, where real-world conditions are more varied and complex, similar to SDD).

This misalignment can lead to bias in the model's predictions because the features learned during training are tailored to the specific characteristics of the training environment. For instance, a model trained in a university might struggle to generalize to the different lighting, backgrounds, or subject appearances found in a retirement home. This results in performance degradation in the new environment, potentially leading to unreliable or inaccurate predictions.

In real-world applications, this could mean that systems designed to function in diverse environments—like healthcare or public services—might fail to provide equitable and accurate results if trained only on narrow or homogeneous datasets. To mitigate these issues, it's important to ensure diverse and representative data collection and to adopt techniques like domain adaptation or generalization methods to reduce performance gaps and biases across different environments.